

# SISTEMA DE GESTIÓN DE SEGUROS DE VIAJE

---

## MANUAL TÉCNICO

SeguroViaje API — Laravel 8 + Frontend HTML

**Autor:**

Luis Angel Chuquimia Choquevilca

Versión 1.0 | 2026

# TABLA DE CONTENIDO

|                                       |    |
|---------------------------------------|----|
| TABLA DE CONTENIDO .....              | 2  |
| 1. Introducción .....                 | 5  |
| 1.1 Objetivo del Sistema.....         | 5  |
| 1.2 Alcance .....                     | 5  |
| 2. Arquitectura del Sistema .....     | 6  |
| 2.1 Stack Tecnológico .....           | 6  |
| 2.2 Estructura de la API.....         | 6  |
| 2.3 Control de Acceso por Roles ..... | 6  |
| 3. Base de Datos .....                | 7  |
| 3.1 Tablas del Sistema .....          | 7  |
| 3.2 Numeración de Pólizas.....        | 7  |
| 3.3 Estados de Siniestro.....         | 7  |
| 4. Backend — Controllers Laravel..... | 8  |
| 4.1 UsuarioController .....           | 8  |
| Rutas .....                           | 8  |
| Método: login().....                  | 8  |
| Método: disableToken(\$id).....       | 8  |
| Validaciones en store() .....         | 9  |
| Código del Controller .....           | 9  |
| 4.2 ClienteController.....            | 10 |
| Rutas .....                           | 10 |
| Validaciones .....                    | 10 |
| Código del Controller .....           | 10 |
| 4.3 PlanController.....               | 11 |
| Rutas .....                           | 11 |
| Campos .....                          | 11 |
| Código del Controller .....           | 11 |
| 4.4 CoberturaController.....          | 12 |
| Rutas .....                           | 12 |
| Campos .....                          | 12 |
| 4.5 PlanCoberturaController.....      | 13 |
| Rutas .....                           | 13 |
| Método: getDetalle() .....            | 13 |
| 4.6 CotizacionController .....        | 14 |
| Rutas .....                           | 14 |
| Validaciones en store() .....         | 14 |
| 4.7 PolizaController .....            | 15 |
| Rutas .....                           | 15 |

|                                                                 |    |
|-----------------------------------------------------------------|----|
| Generación de Número de Póliza .....                            | 15 |
| Método: getPolizaPorNumero(\$numero_poliza).....                | 15 |
| 4.8 SiniestroController .....                                   | 16 |
| Rutas .....                                                     | 16 |
| Método: cambiarEstado(\$id) .....                               | 16 |
| Validaciones en store() .....                                   | 16 |
| 4.9 Otros Controllers .....                                     | 17 |
| RolController.....                                              | 17 |
| PaisController .....                                            | 17 |
| CiudadController .....                                          | 17 |
| 5. Frontend — Vistas HTML.....                                  | 18 |
| 5.1 Estructura de Páginas .....                                 | 18 |
| 5.2 Gestión de Sesión (sessionStorage) .....                    | 18 |
| 5.3 Flujo de Autenticación .....                                | 18 |
| 5.4 login.html.....                                             | 20 |
| Características de Diseño .....                                 | 20 |
| Función Principal: handleLogin().....                           | 20 |
| 5.5 index.html — Panel Principal .....                          | 21 |
| Componentes Principales .....                                   | 21 |
| Control de Menú por Rol.....                                    | 21 |
| 5.6 Páginas CRUD (planes, coberturas, usuarios, clientes) ..... | 22 |
| Patrón Estándar CRUD.....                                       | 22 |
| Página: usuarios.html .....                                     | 22 |
| Página: plan_coberturas.html .....                              | 22 |
| 5.7 siniestros.html .....                                       | 24 |
| Flujo de Uso .....                                              | 24 |
| Endpoint de búsqueda.....                                       | 24 |
| Actualización de Estado.....                                    | 24 |
| 6. Instalación y Configuración .....                            | 25 |
| 6.1 Requisitos del Sistema .....                                | 25 |
| 6.2 Instalación del Backend (Laravel).....                      | 25 |
| 6.3 Configuración del Frontend .....                            | 25 |
| 6.4 Verificación con Thunder Client.....                        | 26 |
| 7. Resumen Completo de Endpoints API .....                      | 27 |
| 8. Notas Técnicas y Consideraciones .....                       | 29 |
| 8.1 Seguridad.....                                              | 29 |
| 8.2 CORS.....                                                   | 29 |
| 8.3 Consideraciones de Producción .....                         | 29 |
| 8.4 Estructura de Archivos Backend.....                         | 29 |



# 1. Introducción

---

El sistema SeguroViaje es una aplicación web de gestión de seguros de viaje desarrollada con Laravel 8 como backend REST API y un frontend en HTML/CSS/JavaScript puro. El sistema permite administrar pólizas, clientes, coberturas, planes, cotizaciones y siniestros, con un control de acceso basado en roles.

## 1.1 Objetivo del Sistema

Proveer una plataforma centralizada para la gestión completa del ciclo de vida de seguros de viaje: desde la cotización hasta el registro y resolución de siniestros, con control de acceso diferenciado por roles de usuario.

## 1.2 Alcance

- Gestión de entidades: países, ciudades, clientes, planes, coberturas, usuarios, roles
- Flujo completo: cotización → póliza → siniestro
- Autenticación con tokens (Laravel Sanctum)
- Control de acceso por roles (4 niveles)
- Frontend HTML con consumo de API REST

## 2. Arquitectura del Sistema

### 2.1 Stack Tecnológico

| Capa      | Tecnología         | Descripción                           |
|-----------|--------------------|---------------------------------------|
| Backend   | Laravel 8          | Framework PHP para la API REST        |
| Auth      | Laravel Sanctum    | Tokens de autenticación stateless     |
| BD        | MySQL / phpMyAdmin | Base de datos relacional (XAMPP)      |
| Frontend  | HTML + JS + CSS    | Interfaz de usuario sin frameworks JS |
| HTTP      | Fetch API          | Comunicación cliente-servidor         |
| Bootstrap | v5.3               | Estilos en páginas de gestión         |

### 2.2 Estructura de la API

La API corre en `http://127.0.0.1:8000/api` y expone endpoints RESTful protegidos con Bearer Token (excepto el login). Todos los endpoints retornan JSON.

### 2.3 Control de Acceso por Roles

| ID Rol | Nombre        | Permisos                                                     |
|--------|---------------|--------------------------------------------------------------|
| 1      | Administrador | Acceso total a todas las secciones del sistema               |
| 2      | Configuración | Solo accede a: Usuarios, Coberturas, Planes, Plan-Coberturas |
| 3      | Ejecutivo     | Accede a: Clientes, Cotizaciones, Pólizas (sin Sinistros)    |
| 4      | Sinistros     | Solo accede al módulo de Sinistros                           |

## 3. Base de Datos

### 3.1 Tablas del Sistema

| Tabla                       | Descripción                                                    |
|-----------------------------|----------------------------------------------------------------|
| <code>pais</code>           | Catálogo de países disponibles                                 |
| <code>ciudad</code>         | Ciudades asociadas a un país (FK: id_pais)                     |
| <code>plan</code>           | Planes de seguro con precio y límite de edad                   |
| <code>cobertura</code>      | Tipos de coberturas con monto máximo                           |
| <code>plan_cobertura</code> | Relación N:M entre planes y coberturas (monto_cubierto)        |
| <code>cliente</code>        | Datos del asegurado (nombre, email, fecha_nacimiento, id_pais) |
| <code>usuario</code>        | Usuarios del sistema con rol y contraseña en texto plano       |
| <code>cotizacion</code>     | Cotización: cliente + plan + ciudades origen/destino + fechas  |
| <code>poliza</code>         | Póliza emitida desde una cotización (número auto-generado)     |
| <code>siniestro</code>      | Siniestro asociado a póliza y cobertura (campo: aprobado 0-3)  |

### 3.2 Numeración de Pólizas

Las pólizas se generan automáticamente con el formato POL-{AÑO}-{NRO}, donde el número es secuencial con padding de 3 dígitos. Ejemplo: POL-2026-001, POL-2026-002, etc.

### 3.3 Estados de Siniestro

| Valor | Estado     | Descripción                              |
|-------|------------|------------------------------------------|
| 0     | Reclamado  | Estado inicial al registrar el siniestro |
| 1     | En Proceso | Siniestro bajo evaluación                |
| 2     | Aprobado   | Siniestro aceptado y aprobado para pago  |
| 3     | Rechazado  | Siniestro denegado                       |

## 4. Backend — Controllers Laravel

Todos los controllers se encuentran en `app/Http/Controllers/` y siguen el patrón RESTful estándar (`index`, `store`, `show`, `update`, `destroy`). La autenticación se maneja mediante `middleware auth:sanctum` en las rutas.

### 4.1 UsuarioController

Gestiona usuarios del sistema, autenticación y manejo de tokens.

#### Rutas

| Método | Ruta                            | Descripción                          |
|--------|---------------------------------|--------------------------------------|
| GET    | /api/usuarios                   | Listar todos los usuarios (con rol)  |
| POST   | /api/usuarios                   | Crear nuevo usuario                  |
| GET    | /api/usuarios/{id}              | Obtener usuario por ID               |
| PUT    | /api/usuarios/{id}              | Actualizar usuario                   |
| DELETE | /api/usuarios/{id}              | Eliminar usuario                     |
| POST   | /api/login                      | Login — devuelve Bearer Token        |
| PUT    | /api/usuario/{id}/disable-token | Revocar todos los tokens del usuario |

#### Método: login()

Valida email y password (texto plano). Si las credenciales son correctas, genera un token Sanctum y retorna los datos del usuario con su rol.

```
POST /api/login
Body: { email, password }

Response 200:
{
  "message": "Login correcto",
  "access_token": "<token>",
  "token_type": "Bearer",
  "usuario": { id_usuario, nombre, email, id_rol, rol: { rol } }
}
```

#### Método: disableToken(\$id)

Elimina todos los tokens Sanctum asociados al usuario. Se llama al hacer logout desde el frontend.

```
PUT /api/usuario/{id}/disable-token
Headers: Authorization: Bearer <token>

Response 200:
{ "message": "Tokens de acceso revocados correctamente para el usuario: ..." }
```



## Validaciones en store()

- nombre: requerido, string, máx 100 chars
- email: requerido, email válido, único en tabla usuario
- password: requerido, mínimo 6 caracteres
- activo: requerido, boolean
- id\_rol: requerido, debe existir en tabla rol

## Código del Controller

```
<?php
namespace App\Http\Controllers;

use App\Models\Usuario;
use Illuminate\Http\Request;

class UsuarioController extends Controller
{
    public function login(Request $request) {
        $request->validate(['email'=>'required|email', 'password'=>'required|string']);
        $usuario = Usuario::with('rol')->where('email',$request->email)->first();
        if (!$usuario || $usuario->password !== $request->password)
            return response()->json(['message'=>'Credenciales incorrectas'],401);
        $token = $usuario->createToken('auth_token')->plainTextToken;
        return response()->json([
            'message'=>'Login correcto',
            'access_token'=>$token,
            'token_type'=>'Bearer',
            'usuario'=>$usuario
        ], 200);
    }

    public function disableToken($id) {
        $usuario = Usuario::find($id);
        if (!$usuario) return response()->json(['message'=>'No encontrado'],404);
        $usuario->tokens()->delete();
        return response()->json(['message'=>'Tokens revocados'],200);
    }
}
```

## 4.2 ClienteController

CRUD completo para clientes asegurados. Incluye relación con la tabla pais.

### Rutas

| Método | Ruta               | Descripción                          |
|--------|--------------------|--------------------------------------|
| GET    | /api/clientes      | Listar todos los clientes (con país) |
| POST   | /api/clientes      | Registrar nuevo cliente              |
| GET    | /api/clientes/{id} | Obtener cliente por ID               |
| PUT    | /api/clientes/{id} | Actualizar cliente                   |
| DELETE | /api/clientes/{id} | Eliminar cliente                     |

### Validaciones

- nombre, apellido: requerido, string, máx 100
- email: requerido, email válido, único (excepto en update del mismo registro)
- telefono: nullable, string, máx 20
- fecha\_nacimiento: requerido, date
- id\_pais: requerido, debe existir en tabla pais

### Código del Controller

```
public function store(Request $request) {  
    $request->validate([  
        'nombre' => 'required|string|max:100',  
        'apellido' => 'required|string|max:100',  
        'email' => 'required|email|unique:cliente,email',  
        'telefono' => 'nullable|string|max:20',  
        'fecha_nacimiento' => 'required|date',  
        'id_pais' => 'required|exists:pais,id_pais',  
    ]);  
    $cliente = Cliente::create($request->all());  
    return response()->json($cliente, 201);  
}
```

## 4.3 PlanController

Gestión de planes de seguro. Incluye relación con coberturas a través de la tabla plan\_cobertura.

### Rutas

| Método | Ruta             | Descripción                              |
|--------|------------------|------------------------------------------|
| GET    | /api/planes      | Listar planes (con coberturas asociadas) |
| POST   | /api/planes      | Crear nuevo plan                         |
| GET    | /api/planes/{id} | Obtener plan por ID                      |
| PUT    | /api/planes/{id} | Actualizar plan                          |
| DELETE | /api/planes/{id} | Eliminar plan                            |

### Campos

- nombre: string, requerido, máx 100
- descripcion: nullable, texto libre
- precio: decimal, requerido, mínimo 0
- limite\_edad: entero, requerido, mínimo 1

### Código del Controller

```
public function index() {
    $planes = Plan::with('coberturas')->get();
    return response()->json($planes, 200);
}

public function store(Request $request) {
    $request->validate([
        'nombre' => 'required|string|max:100',
        'descripcion' => 'nullable|string',
        'precio' => 'required|numeric|min:0',
        'limite_edad' => 'required|integer|min:1',
    ]);
    $plan = Plan::create($request->all());
    return response()->json($plan, 201);
}
```

## 4.4 CoberturaController

CRUD para los tipos de coberturas ofrecidas por la aseguradora.

### Rutas

| Método | Ruta                 | Descripción                 |
|--------|----------------------|-----------------------------|
| GET    | /api/coberturas      | Listar todas las coberturas |
| POST   | /api/coberturas      | Crear nueva cobertura       |
| GET    | /api/coberturas/{id} | Obtener cobertura por ID    |
| PUT    | /api/coberturas/{id} | Actualizar cobertura        |
| DELETE | /api/coberturas/{id} | Eliminar cobertura          |

### Campos

- nombre: string, requerido, máx 100
- descripcion: nullable, texto libre
- monto\_maximo: decimal, requerido, mínimo 0

## 4.5 PlanCoberturaController

Gestiona la relación N:M entre planes y coberturas, con monto específico cubierto por plan.

### Rutas

| Método | Ruta                                | Descripción                            |
|--------|-------------------------------------|----------------------------------------|
| GET    | /api/plan-coberturas                | Listar todas las relaciones            |
| GET    | /api/plan-coberturas-detalle        | Listar con nombres de plan y cobertura |
| GET    | /api/plan-coberturas/plan/{id_plan} | Coberturas disponibles de un plan      |
| POST   | /api/plan-coberturas                | Crear relación plan-cobertura          |
| PUT    | /api/plan-coberturas/{id}           | Actualizar relación                    |
| DELETE | /api/plan-coberturas/{id}           | Eliminar relación                      |

### Método: getDetalle()

Realiza un JOIN entre plan\_cobertura, plan y cobertura para retornar la relación con los nombres descriptivos.

```
public function getDetalle() {
    $resultados = DB::table('plan_cobertura')
        ->join('plan', 'plan_cobertura.id_plan', '=', 'plan.id_plan')
        -
        >join('cobertura', 'plan_cobertura.id_cobertura', '=', 'cobertura.id_cobertura')
        ->select(
            'plan_cobertura.id',
            'plan_cobertura.id_plan',
            'plan.nombre',
            'plan_cobertura.id_cobertura',
            'cobertura.nombre as nombrecobertura',
            'plan_cobertura.monto_cubierto'
        )->get();
    return response()->json($resultados, 200);
}
```

## 4.6 CotizacionController

Gestiona las cotizaciones de seguros de viaje, que vinculan cliente, plan y ciudades de origen/destino.

### Rutas

| Método | Ruta                   | Descripción                                     |
|--------|------------------------|-------------------------------------------------|
| GET    | /api/cotizaciones      | Listar cotizaciones (cliente + plan + ciudades) |
| POST   | /api/cotizaciones      | Crear cotización                                |
| GET    | /api/cotizaciones/{id} | Obtener cotización por ID                       |
| PUT    | /api/cotizaciones/{id} | Actualizar cotización                           |
| DELETE | /api/cotizaciones/{id} | Eliminar cotización                             |

### Validaciones en store()

- id\_cliente: requerido, debe existir en tabla cliente
- id\_plan: requerido, debe existir en tabla plan
- id\_ciudad\_origen, id\_ciudad\_destino: requeridos, deben existir en tabla ciudad
- fecha\_inicio: requerida, formato date
- fecha\_fin: requerida, date, debe ser posterior a fecha\_inicio
- precio\_total: requerido, numeric, mínimo 0
- estado: requerido, uno de: pendiente | aprobada | rechazada

## 4.7 PolizaController

Emita pólizas a partir de cotizaciones aprobadas. Genera el número de póliza automáticamente en formato POL-{AÑO}-{NRO}.

### Rutas

| Método | Ruta                                 | Descripción                         |
|--------|--------------------------------------|-------------------------------------|
| GET    | /api/polizas                         | Listar pólizas (con cotización)     |
| POST   | /api/polizas                         | Emitir nueva póliza (número auto)   |
| GET    | /api/polizas/{id}                    | Obtener póliza por ID               |
| PUT    | /api/polizas/{id}                    | Actualizar póliza                   |
| DELETE | /api/polizas/{id}                    | Eliminar póliza                     |
| GET    | /api/polizas-detalle                 | Pólizas con datos de cliente y plan |
| GET    | /api/polizas-coberturas              | Pólizas con sus coberturas          |
| GET    | /api/polizas-coberturas/{nro_poliza} | Coberturas de una póliza específica |

### Generación de Número de Póliza

```
// Auto-generación del número de póliza
$year = date('Y');
$lastPoliza = Poliza::where('numero_poliza', 'like', "POL-{$year}-%")
    ->orderBy('id_poliza', 'desc')->first();
$nextNumber = ($lastPoliza) ? (int)substr($lastPoliza->numero_poliza, -3)+1 : 1;
$numeroGenerado = 'POL-'. $year . '-' . str_pad($nextNumber, 3, '0', STR_PAD_LEFT);
// Ejemplo: POL-2026-001, POL-2026-002, ...
```

### Método: getPolizaPorNumero(\$numero\_poliza)

Consulta en cadena: poliza → cotizacion → cliente → plan\_cobertura → cobertura. Retorna datos del cliente y todas las coberturas incluidas en la póliza.

```
GET /api/polizas-coberturas/POL-2026-001
Headers: Authorization: Bearer <token>

Response:
[
  { "id_poliza":1, "numero_poliza":"POL-2026-001",
    "nombre":"Juan","apellido":"Pérez",
    "id_cobertura":2,"nombrecobertura":"Asistencia Médica" },
  { "id_poliza":1, "numero_poliza":"POL-2026-001",
    "nombre":"Juan","apellido":"Pérez",
    "id_cobertura":3,"nombrecobertura":"Pérdida de Equipaje" }
]
```

## 4.8 SiniestroController

Gestiona el registro y seguimiento de siniestros. Incluye métodos especiales para consulta por póliza y cambio de estado.

### Rutas

| Método | Ruta                                | Descripción                                |
|--------|-------------------------------------|--------------------------------------------|
| GET    | /api/siniestros                     | Listar siniestros (póliza + cobertura)     |
| POST   | /api/siniestros                     | Registrar nuevo siniestro                  |
| GET    | /api/siniestros/{id}                | Obtener siniestro por ID                   |
| PUT    | /api/siniestros/{id}                | Actualizar siniestro                       |
| DELETE | /api/siniestros/{id}                | Eliminar siniestro                         |
| GET    | /api/siniestros-detalle             | Siniestros con datos de póliza y cobertura |
| GET    | /api/siniestros-poliza/{nro_poliza} | Siniestros de una póliza específica        |
| PUT    | /api/siniestro/{id}/estado          | Cambiar estado del siniestro (0-3)         |

### Método: cambiarEstado(\$id)

Permite actualizar el campo aprobado del siniestro con valores del 0 al 3. Usado desde el dropdown en la vista de siniestros.

```
PUT /api/siniestro/{id}/estado
Body: { aprobado: 0|1|2|3 }
Validación: 'aprobado' => 'required|integer|in:0,1,2,3'

Estados:
  0 = Reclamado
  1 = En Proceso
  2 = Aprobado
  3 = Rechazado
```

### Validaciones en store()

- id\_poliza: requerido, debe existir en tabla poliza
- id\_cobertura: requerido, debe existir en tabla cobertura
- fecha\_siniestro: requerida, date
- descripcion: requerida, string
- monto\_reclamado: requerido, numeric, mínimo 0
- aprobado: requerido, boolean (se envía 0 al registrar)



## 4.9 Otros Controllers

### RolController

CRUD para roles del sistema. Los roles son: Administrador (1), Configuración (2), Ejecutivo (3), Siniestros (4).

| Método | Ruta            | Descripción            |
|--------|-----------------|------------------------|
| GET    | /api/roles      | Listar todos los roles |
| POST   | /api/roles      | Crear rol              |
| GET    | /api/roles/{id} | Obtener rol por ID     |
| PUT    | /api/roles/{id} | Actualizar rol         |
| DELETE | /api/roles/{id} | Eliminar rol           |

### PaisController

Catálogo de países. Usado como FK en clientes y ciudades.

| Método | Ruta             | Descripción         |
|--------|------------------|---------------------|
| GET    | /api/paises      | Listar países       |
| POST   | /api/paises      | Crear país          |
| GET    | /api/paises/{id} | Obtener país por ID |
| PUT    | /api/paises/{id} | Actualizar país     |
| DELETE | /api/paises/{id} | Eliminar país       |

### CiudadController

Ciudades con FK al país. Usadas en cotizaciones como ciudad de origen y destino.

| Método | Ruta               | Descripción                          |
|--------|--------------------|--------------------------------------|
| GET    | /api/ciudades      | Listar ciudades (con datos del país) |
| POST   | /api/ciudades      | Crear ciudad                         |
| GET    | /api/ciudades/{id} | Obtener ciudad por ID                |
| PUT    | /api/ciudades/{id} | Actualizar ciudad                    |
| DELETE | /api/ciudades/{id} | Eliminar ciudad                      |

## 5. Frontend — Vistas HTML

El frontend está compuesto por páginas HTML independientes que consumen la API REST mediante Fetch. Las sesiones se gestionan con sessionStorage del navegador.

### 5.1 Estructura de Páginas

| Archivo                           | Descripción                                                  |
|-----------------------------------|--------------------------------------------------------------|
| <code>login.html</code>           | Pantalla de inicio de sesión con diseño animado split-panel  |
| <code>index.html</code>           | Panel principal con estadísticas, menú hamburguesa y sidebar |
| <code>usuarios.html</code>        | Gestión CRUD de usuarios con selector de rol                 |
| <code>planes.html</code>          | Gestión CRUD de planes de seguro                             |
| <code>coberturas.html</code>      | Gestión CRUD de coberturas                                   |
| <code>plan_coberturas.html</code> | Relación Plan-Cobertura con autocompletado de monto          |
| <code>clientes.html</code>        | Gestión CRUD de clientes asegurados                          |
| <code>cotizaciones.html</code>    | Registro y gestión de cotizaciones                           |
| <code>polizas.html</code>         | Emisión y gestión de pólizas                                 |
| <code>siniestros.html</code>      | Registro y seguimiento de siniestros por número de póliza    |

### 5.2 Gestión de Sesión (sessionStorage)

Al hacer login exitoso, se guardan los siguientes datos en sessionStorage:

| Clave                      | Contenido                                          |
|----------------------------|----------------------------------------------------|
| <code>access_token</code>  | Token Bearer para autenticar las requests a la API |
| <code>token_type</code>    | Tipo de token (siempre 'Bearer')                   |
| <code>idUsuario</code>     | ID del usuario logueado                            |
| <code>nombreUsuario</code> | Nombre del usuario para mostrar en UI              |
| <code>emailUsuario</code>  | Email del usuario                                  |
| <code>idRol</code>         | ID del rol para control de menú                    |
| <code>rol</code>           | Nombre del rol                                     |
| <code>usuario</code>       | Objeto JSON completo del usuario                   |

### 5.3 Flujo de Autenticación

```
// 1. Login → guardar token
sessionStorage.setItem('access_token', data.access_token);
window.location.href = 'index.html';
```

```
// 2. En cada página, verificar sesión
const token = sessionStorage.getItem('access_token');
if (!token) window.location.href = 'login.html';

// 3. Incluir token en cada request
fetch(url, { headers: {
  'Authorization': `Bearer ${token}`,
  'Content-Type': 'application/json'
}})

// 4. Logout → revocar token y limpiar sesión
await fetch(`/api/usuario/${idUsuario}/disable-token`, { method: 'PUT', ... });
sessionStorage.clear();
window.location.href = 'login.html';
```

## 5.4 login.html

Página de inicio de sesión con diseño split-panel: lado izquierdo decorativo con animación de globo terráqueo SVG y lado derecho con el formulario de login.

### Características de Diseño

- Paleta oscura: --navy #0F1B2D, --teal #3BC4A0, --blue #1E6FBF
- Animación de globo SVG con rotación continua (22s)
- Puntos flotantes decorativos con keyframes float
- Botón con estado loading (spinner animado)
- Validación en cliente antes de enviar
- Responsive: oculta panel izquierdo en móvil

### Función Principal: handleLogin()

```
async function handleLogin() {
  const email = document.getElementById('email').value.trim();
  const password = document.getElementById('password').value.trim();

  btn.classList.add('loading'); // muestra spinner

  const response = await fetch(`${API_BASE}/login`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email, password })
  });

  const data = await response.json();
  if (!response.ok) { showError(data.message); return; }

  sessionStorage.setItem('access_token', data.access_token);
  sessionStorage.setItem('idRol', data.usuario.id_rol);
  // ... guardar más datos de sesión ...
  window.location.href = 'index.html';
}
```

## 5.5 index.html — Panel Principal

Dashboard con topbar fijo, sidebar lateral deslizante y hero de bienvenida personalizado.

### Componentes Principales

- Topbar: logo, chip de usuario con inicial, botón de logout
- Sidebar: menú jerárquico con grupos colapsables (Configuración y Gestión)
- Menú hamburguesa: toggle del sidebar con clase CSS sidebar-open
- Welcome Hero: saludo personalizado con reloj en tiempo real
- Stats Cards: 4 tarjetas con métricas del sistema (valores estáticos de muestra)
- Panel de sesión: FAB flotante que muestra variables de sessionStorage (solo desarrollo)
- Sección de Documentación: links para descargar Manual de Usuario y Manual Técnico

### Control de Menú por Rol

```
const idRolMenu = Number(sessionStorage.getItem('idRol'));

if (idRolMenu === 2) { // Solo Configuración
  // Ocultar: Clientes, Pólizas, Cotizaciones, Siniestros
}
else if (idRolMenu === 3) { // Ejecutivo
  // Ocultar: Configuración, Siniestros
}
else if (idRolMenu === 4) { // Siniestros
  // Ocultar: Configuración, Clientes, Pólizas, Cotizaciones
}
// Rol 1 (Admin): ve todo
```

## 5.6 Páginas CRUD (planes, coberturas, usuarios, clientes)

Las páginas de gestión siguen el mismo patrón: formulario de captura en card superior + tabla con datos + botones de editar/eliminar. Usan Bootstrap 5.3 para los estilos.

### Patrón Estándar CRUD

```
// 1. Cargar registros al iniciar
async function cargarRegistros() {
  const res = await fetch(API_URL, {
    headers: { 'Authorization': `Bearer ${token}` }
  });
  const data = await res.json();
  tabla.innerHTML = data.map(item => `<tr>...</tr>`).join('');
}

// 2. Guardar (POST o PUT según si hay ID)
async function guardarRegistro() {
  const id = document.getElementById('id').value;
  const method = id ? 'PUT' : 'POST';
  const url = id ? `${API_URL}/${id}` : API_URL;
  await fetch(url, { method, headers: {...}, body: JSON.stringify(data) });
  limpiarForm(); cargarRegistros();
}

// 3. Editar → prellenar formulario
function editarRegistro(id, campo1, campo2, ...) {
  document.getElementById('id').value = id;
  document.getElementById('campo1').value = campo1;
}

// 4. Eliminar con confirmación
async function eliminarRegistro(id) {
  if (confirm('¿Eliminar?')) {
    await fetch(`${API_URL}/${id}`, { method: 'DELETE', headers: {...} });
    cargarRegistros();
  }
}
```

### Página: usuarios.html

Tiene un selector de rol con opciones legibles: Administrador (1), Configuración (2), Ejecutivo (3), Siniestros (4). Al guardar, el value del select ya es el ID numérico correcto.

### Página: plan\_coberturas.html

Funcionalidad especial: al seleccionar un plan, carga dinámicamente las coberturas disponibles para ese plan. Al seleccionar una cobertura, autocompleta el monto\_cubierto con el monto\_maximo de la cobertura.

```
// Cargar coberturas al cambiar plan
document.getElementById('id_plan').addEventListener('change', function() {
```

```
    cargarCoberturasDelPlan(this.value);
  });

// Autocompletar monto al seleccionar cobertura
document.getElementById('id_cobertura').addEventListener('change', function() {
  const item = coberturasDisponibles.find(c => c.id_cobertura == this.value);
  if (item) document.getElementById('monto_cubierto').value = item.monto_maximo;
});
```

## 5.7 siniestros.html

Página especializada para búsqueda por número de póliza, registro de siniestros y actualización de estado. No sigue el patrón CRUD estándar.

### Flujo de Uso

- 1. Ingresar número de póliza (formato POL-2026-001) y pulsar Buscar
- 2. La API retorna datos del cliente y coberturas incluidas en la póliza
- 3. Se muestra alerta con nombre del cliente y número de póliza
- 4. Tabla con coberturas disponibles y botón 'Registrar Siniestro' por cobertura
- 5. Al pulsar el botón, aparece formulario con fecha, monto y descripción
- 6. Al confirmar, se envía POST /api/siniestros con aprobado: 0
- 7. El reporte inferior muestra todos los siniestros de esa póliza
- 8. Dropdown por siniestro para cambiar estado (Reclamado/En Proceso/Aprobado/Rechazado)

### Endpoint de búsqueda

```
GET /api/polizas-coberturas/{numero_poliza}
```

Retorna array con:

```
- id_poliza, numero_poliza, fecha_vencimiento
- nombre, apellido del cliente
- id_cobertura, nombrecobertura
(una fila por cobertura incluida en el plan de la póliza)
```

### Actualización de Estado

```
async function actualizarEstadoSiniestro(idSiniestro, nuevoEstado, nroPoliza) {
  await fetch(`/api/siniestro/${idSiniestro}/estado`, {
    method: 'PUT',
    headers: { 'Authorization': `Bearer ${token}`, ... },
    body: JSON.stringify({ aprobado: parseInt(nuevoEstado, 10) })
  });
  // Recargar reporte después de actualizar
  cargarReporteSiniestros(nroPoliza, token);
}
```



## 6. Instalación y Configuración

---

### 6.1 Requisitos del Sistema

- PHP 8.0 o superior
- Composer 2.x
- XAMPP (Apache + MySQL) o equivalente
- Node.js (opcional, para desarrollo frontend)
- Navegador web moderno (Chrome, Firefox, Edge)

### 6.2 Instalación del Backend (Laravel)

```
# 1. Clonar o descomprimir el proyecto
cd C:/xampp/htdocs/seguroviaje_api

# 2. Instalar dependencias PHP
composer install

# 3. Configurar entorno
cp .env.example .env
# Editar .env:
# DB_DATABASE=seguroviaje_db
# DB_USERNAME=root
# DB_PASSWORD=

# 4. Generar clave de aplicación
php artisan key:generate

# 5. Ejecutar migraciones y seeders
php artisan migrate --seed

# 6. Instalar Sanctum
php artisan vendor:publish --provider='Laravel\Sanctum\SanctumServiceProvider'

# 7. Iniciar servidor
php artisan serve
# API disponible en: http://127.0.0.1:8000/api
```

### 6.3 Configuración del Frontend

El frontend no requiere instalación. Basta con abrir los archivos HTML directamente en el navegador o servirlos desde un servidor HTTP simple.

```
# Opción 1: Abrir directamente
# Doble clic en login.html

# Opción 2: Servidor simple con Python
cd frontend/
python -m http.server 3000
```

```
# Abrir: http://localhost:3000/login.html

# Opción 3: Servidor simple con Node
npx serve . -p 3000
```

## 6.4 Verificación con Thunder Client

Para probar los endpoints antes de conectar el frontend:

```
# 1. Login
POST http://127.0.0.1:8000/api/login
Body: { "email": "admin@test.com", "password": "123456" }

# 2. Copiar access_token de la respuesta

# 3. Usar el token en headers
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI...

# 4. Probar endpoint protegido
GET http://127.0.0.1:8000/api/clientes
Headers: Authorization: Bearer <token>
```

## 7. Resumen Completo de Endpoints API

| Método | Ruta                            | Descripción                      |
|--------|---------------------------------|----------------------------------|
| POST   | /api/login                      | Login — obtener token Bearer     |
| GET    | /api/usuarios                   | Listar usuarios                  |
| POST   | /api/usuarios                   | Crear usuario                    |
| PUT    | /api/usuarios/{id}              | Actualizar usuario               |
| DELETE | /api/usuarios/{id}              | Eliminar usuario                 |
| PUT    | /api/usuario/{id}/disable-token | Revocar token (logout)           |
| GET    | /api/roles                      | Listar roles                     |
| GET    | /api/paises                     | Listar países                    |
| GET    | /api/ciudades                   | Listar ciudades                  |
| GET    | /api/planes                     | Listar planes                    |
| POST   | /api/planes                     | Crear plan                       |
| PUT    | /api/planes/{id}                | Actualizar plan                  |
| DELETE | /api/planes/{id}                | Eliminar plan                    |
| GET    | /api/coberturas                 | Listar coberturas                |
| POST   | /api/coberturas                 | Crear cobertura                  |
| PUT    | /api/coberturas/{id}            | Actualizar cobertura             |
| DELETE | /api/coberturas/{id}            | Eliminar cobertura               |
| GET    | /api/plan-coberturas            | Listar relaciones plan-cobertura |
| GET    | /api/plan-coberturas-detalle    | Con nombres descriptivos         |
| GET    | /api/plan-coberturas/plan/{id}  | Coberturas de un plan            |
| POST   | /api/plan-coberturas            | Crear relación                   |
| PUT    | /api/plan-coberturas/{id}       | Actualizar relación              |
| DELETE | /api/plan-coberturas/{id}       | Eliminar relación                |
| GET    | /api/clientes                   | Listar clientes                  |
| POST   | /api/clientes                   | Crear cliente                    |
| PUT    | /api/clientes/{id}              | Actualizar cliente               |
| DELETE | /api/clientes/{id}              | Eliminar cliente                 |
| GET    | /api/cotizaciones               | Listar cotizaciones              |
| POST   | /api/cotizaciones               | Crear cotización                 |
| PUT    | /api/cotizaciones/{id}          | Actualizar cotización            |
| DELETE | /api/cotizaciones/{id}          | Eliminar cotización              |
| GET    | /api/polizas                    | Listar pólizas                   |
| POST   | /api/polizas                    | Emitir póliza (número auto)      |

|               |                               |                                 |
|---------------|-------------------------------|---------------------------------|
| <b>PUT</b>    | /api/polizas/{id}             | Actualizar póliza               |
| <b>DELETE</b> | /api/polizas/{id}             | Eliminar póliza                 |
| <b>GET</b>    | /api/polizas-detalle          | Pólizas con cliente y plan      |
| <b>GET</b>    | /api/polizas-coberturas       | Pólizas con coberturas          |
| <b>GET</b>    | /api/polizas-coberturas/{nro} | Coberturas de póliza por número |
| <b>GET</b>    | /api/siniestros               | Listar siniestros               |
| <b>POST</b>   | /api/siniestros               | Registrar siniestro             |
| <b>PUT</b>    | /api/siniestros/{id}          | Actualizar siniestro            |
| <b>DELETE</b> | /api/siniestros/{id}          | Eliminar siniestro              |
| <b>GET</b>    | /api/siniestros-detalle       | Siniestros con detalles         |
| <b>GET</b>    | /api/siniestros-poliza/{nro}  | Siniestros de una póliza        |
| <b>PUT</b>    | /api/siniestro/{id}/estado    | Cambiar estado (0-3)            |

## 8. Notas Técnicas y Consideraciones

---

### 8.1 Seguridad

- Las contraseñas se almacenan en texto plano (sin hash). Para producción se debe implementar bcrypt o Argon2.
- Los tokens Sanctum se revocan correctamente al hacer logout mediante PUT /api/usuario/{id}/disable-token.
- Todos los endpoints (excepto /api/login) requieren Authorization: Bearer <token>.

### 8.2 CORS

Para permitir que el frontend (en un dominio o puerto diferente) consuma la API, verificar la configuración de CORS en config/cors.php y que sanctum.stateful en .env incluya el origen correcto.

### 8.3 Consideraciones de Producción

- Cambiar APP\_ENV=production y APP\_DEBUG=false en .env
- Usar HTTPS para proteger los tokens en tránsito
- Implementar hash de contraseñas (Hash::make en store/update de UsuarioController)
- Configurar rate limiting en api.php para proteger el endpoint /api/login
- Considerar almacenar el token en httpOnly cookie en lugar de sessionStorage

### 8.4 Estructura de Archivos Backend

```
seguroviaje_api/
├── app/
│   ├── Http/Controllers/
│   │   ├── UsuarioController.php
│   │   ├── ClienteController.php
│   │   ├── PlanController.php
│   │   ├── CoberturaController.php
│   │   ├── PlanCoberturaController.php
│   │   ├── CotizacionController.php
│   │   ├── PolizaController.php
│   │   ├── SiniestroController.php
│   │   ├── RolController.php
│   │   ├── PaisController.php
│   │   └── CiudadController.php
│   ├── Models/
│   │   ├── Usuario.php
│   │   ├── Cliente.php
│   │   ├── Plan.php
│   │   └── ... (un modelo por tabla)
├── database/
│   ├── migrations/      (10 migraciones)
│   └── seeders/         (datos de prueba)
```

```
└─ routes/  
  └─ api.php      (definición de rutas)
```